UrbanGuardian AI- Smart Citizen Grievence Detection with Kafka Based Resolution and Automated Escalation

A Mini Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

Theegala Dhanya Sri – 2420030338

Nidumolu Veda Samhita – 2420030468

Bhagyalaxmi Murude – 2420030700

Under the Esteemed Guidance of

Dr.Neelima Gogineni

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

The Mini Project Report entitled **"UrbanGuardian AI – Smart Citizen Grievance Detection with Kafka-Based Resolution and Automated Escalation"** is a record of the bona fide work carried out by **Theegala Dhanya Sri 2420030338, Nidumolu Veda Samhita – 2420030468 and Bhagyalaxmi Murude 2420030700**, submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (or) Computer Science and Information Technology** at **K L University**. The results embodied in this report have not been copied from any other department, university, or institute.

Nidumolu Veda Samhita – 2420030468

Theegala Dhanya Sri – 2420030338

Bhagyalaxmi Murude – 2420030700

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **UrbanGuardian AI – Smart Citizen Grievance Detection with Kafka Based Resolution and Automated Escalation** is a bonafide work done and submitted by **Theegala Dhanya Sri - 2420030338, Nidumolu Veda Samhita - 2420030468, Bhagyalaxmi Murude- 2420030700** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Course Co-ordinator **Signature of HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Dr.Neelima Gogineni** , Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Theegala Dhanya Sri – 2420030338

Nidumolu Veda Samhita – 2420030468

Bhagyalaxmi Murude – 2420030700

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8-10
4.	Problem Statement and Objectives	11
5.	Hardware and Software requirements	12
6.	Implementation	13-19
7.	Results & Discussion	21-26
8.	Conclusion & Future Scope	27
9.	References	28

1. ABSTRACT

Citizen grievances related to roads, sanitation, waste management, water supply, and public infrastructure are often managed through traditional systems involving manual categorization, department assignment, and resolution tracking. Citizens face difficulties in reporting and tracking issues, while authorities struggle with prioritization, coordination, and delayed resolutions. These challenges highlight the need for an intelligent and automated grievance management system.

To address these challenges, **UrbanGuardian AI – Smart Citizen Grievance Detection with Kafka-Based Resolution and Automated Escalation** is developed as a service-oriented platform for managing citizen complaints. Citizens can submit grievances through a unified interface, while AI identifies categories, priorities, and responsible departments. **React.js** supports the frontend, while **Spring Boot and Spring Cloud** provide the microservices-based backend.

The **first approach** focuses on **AI-based grievance classification and priority analysis** using AI and Large Language Model (LLM) technologies. Citizens describe issues in natural language, and the analysis service determines their category and urgency, reducing manual effort and improving processing efficiency.

The **second approach** focuses on **intelligent department routing and microservices coordination**. Complaints are assigned to appropriate departments based on their category and priority. **Eureka Service Discovery** enables service registration, while **Spring Cloud Gateway** supports API routing and communication between services.

The **third approach** addresses **event-driven complaint monitoring and automated escalation** using Apache Kafka. Complaint creation and status updates generate events, while unresolved complaints are automatically escalated based on predefined conditions, improving accountability and reducing delays.

The platform uses **PostgreSQL** for data storage and **REST APIs** for service integration. Administrative dashboards support complaint monitoring and analytics, while an AI chatbot assists citizens with grievance-related queries. Overall, **UrbanGuardian AI** combines **AI-assisted complaint analysis, microservices architecture, service discovery, Kafka-based event processing, and automated escalation** to improve departmental coordination, enhance transparency, and provide efficient civic grievance management.

Team Members:

1. Theegala Dhanya Sri – 2420030338
2. Nidumolu Veda Samhita – 2420030468
3. Bhagyalaxmi Murude – 2420030700

2. INTRODUCTION

The rapid growth of urban populations has increased the challenges associated with managing civic issues such as improper waste disposal, damaged roads, water supply disruptions, drainage problems, and malfunctioning streetlights. Traditional grievance management systems often depend on manual complaint registration, classification, and departmental assignment, resulting in delayed resolutions, inefficient communication, and limited transparency. Citizens also face difficulties in tracking complaints and receiving timely updates from municipal authorities.

UrbanGuardian AI – Smart Citizen Grievance Detection with Kafka-Based Resolution and Automated Escalation is proposed as an intelligent civic grievance management platform that simplifies complaint registration, classification, tracking, and resolution management. The system enables citizens to report civic issues through a web-based interface and monitor their progress using unique complaint tracking identifiers. It aims to improve communication between citizens and municipal authorities while reducing manual administrative effort.

The platform incorporates an **AI-Powered Complaint Analysis Engine** that analyzes complaint descriptions to automatically identify issue categories, determine priority levels, and recommend responsible departments. An **Administrative Dashboard** allows authorized personnel to review complaints, update their status, and monitor resolution progress. An **AI-Assisted Chatbot** provides interactive guidance to citizens regarding complaint registration and civic services. The proposed **Automated Escalation Mechanism** is designed to identify delayed or unresolved grievances and support timely administrative intervention.

From a software architecture perspective, UrbanGuardian AI follows a **Service-Oriented Architecture (SOA) using microservices**, developed with **Java, Spring Boot, and Spring Cloud**. Independent backend services support complaint management, AI analysis, and service coordination. **Eureka Service Discovery** enables service registration and discovery, while **Spring Cloud Gateway** provides a centralized entry point for API requests. **PostgreSQL** supports persistent complaint storage, and **REST APIs** facilitate communication between services. **Apache Kafka** is included in the proposed architecture to support asynchronous event-driven communication and complaint workflow coordination.

The frontend is developed using **React.js, JavaScript, HTML, and CSS**, providing accessible interfaces for citizens and administrators. **Python and Google Gemini AI** support intelligent grievance analysis and conversational assistance.

Overall, **UrbanGuardian AI** aims to transform traditional civic grievance management into an **intelligent, transparent, and efficient digital workflow**. By combining AI-assisted complaint classification, microservices, centralized API communication, complaint tracking, and proposed event-driven escalation, the platform provides a scalable foundation for improving public service delivery, strengthening citizen participation, and supporting responsive urban governance.

3. LITERATURE SURVEY

3.1 Smart City Grievance Redressal System

Smart city grievance redressal systems provide digital platforms through which citizens can report civic problems such as damaged roads, overflowing garbage, water supply disruptions, drainage issues, and non-functional streetlights. These systems help citizens submit complaints and enable municipal authorities to monitor and manage reported issues. However, UrbanGuardian AI aims to enhance conventional grievance management by incorporating AI-based grievance classification, automated departmental routing, and systematic escalation of unresolved complaints.

3.2 Municipal Complaint Management Platforms

Municipal complaint management platforms allow citizens to register civic complaints and track their progress through online portals or mobile applications. These platforms help local authorities organize complaints, assign them to relevant departments, and update their resolution status. However, UrbanGuardian AI focuses on improving complaint processing through intelligent grievance detection and Kafka-based event communication, supporting efficient coordination between system components.

3.3 Swachhata-MoHUA – Civic Sanitation Grievance Platform

Swachhata-MoHUA is a civic grievance platform associated with urban sanitation and cleanliness. It enables citizens to report issues related to garbage collection, public cleanliness, and other sanitation concerns. Such platforms demonstrate the importance of citizen participation and digital complaint tracking in maintaining urban environments. However, UrbanGuardian AI proposes a broader grievance management approach that can classify different categories of civic issues and support automated routing and escalation workflows.

3.4 MyGov – Citizen Engagement Platform

MyGov is a citizen engagement platform that facilitates communication between citizens and government initiatives through discussions, participation activities, and feedback mechanisms. It demonstrates the role of digital platforms in strengthening communication between the public and government institutions. However, UrbanGuardian AI focuses specifically on civic grievance processing by organizing reported issues, directing complaints to appropriate departments, and tracking their resolution through a structured workflow.

3.5 CPGRAMS – Public Grievance Redressal System

The Centralized Public Grievance Redress and Monitoring System (CPGRAMS) is an online platform used for registering and monitoring public grievances related to government departments and organizations. It provides a structured mechanism for grievance submission and status tracking. However, UrbanGuardian AI aims to enhance the handling of civic complaints through AI-assisted classification, event-driven processing using Apache Kafka, and automated escalation of complaints that remain unresolved beyond defined timelines.

3.6 Integrated Command and Control Centre Systems

Integrated Command and Control Centre systems are used in smart city initiatives to support the monitoring and coordination of urban services. They may bring together information from different civic departments and operational systems to improve decision-making and response coordination. However, UrbanGuardian AI focuses on the citizen grievance lifecycle, providing a centralized process for reporting issues, classifying complaints, routing them to the appropriate department, and monitoring resolution progress.

3.7 Online Civic Feedback and Reporting Platforms

Online civic feedback platforms allow residents to communicate local problems and provide information about public infrastructure and municipal services. They improve accessibility by allowing citizens to report issues without depending entirely on physical visits to government offices. However, UrbanGuardian AI aims to extend this approach through AI-based grievance identification and structured complaint management, reducing the need for manual categorization and supporting more consistent processing.

3.8 Mobile-Based Public Service Applications

Mobile-based public service applications provide citizens with convenient access to government services, complaint registration, notifications, and status updates. These applications help improve communication between citizens and public authorities. However, UrbanGuardian AI emphasizes the backend processing of grievances through Kafka-based event communication, enabling complaint registration, status changes, and escalation events to be handled through an organized workflow.

3.9 Digital Complaint Tracking Systems

Digital complaint tracking systems help organizations record complaints, assign responsibility, monitor progress, and maintain resolution histories. They improve transparency by allowing users or administrators to view the current status of reported issues. However, UrbanGuardian AI aims to combine complaint tracking with AI-assisted classification and automated escalation, helping identify unresolved grievances and directing attention to complaints that require further action.

3.10 Event-Driven Systems for Public Service Management

Event-driven architectures support communication between different application components through events generated when important activities occur. Technologies such as Apache Kafka can be used to process events asynchronously and distribute updates between services. In grievance management, events may represent complaint registration, department assignment, status changes, deadline violations, and escalation actions. UrbanGuardian AI applies this architectural approach to support organized grievance processing and automated escalation of unresolved complaints.

3.11 Research Gap

Existing civic grievance platforms demonstrate the importance of online complaint registration, citizen participation, grievance tracking, and coordination between government departments. However, conventional complaint management workflows may still depend on manual classification, assignment, follow-up, and escalation. These limitations can contribute to delays, inconsistent prioritization, and reduced visibility into unresolved grievances.

UrbanGuardian AI addresses this gap by integrating AI-based grievance detection and classification, departmental complaint routing, Kafka-based event processing, complaint status tracking, and automated escalation into a unified platform. Apache Kafka supports event-driven communication between system components, while the application backend manages grievance processing and resolution workflows. A database maintains complaint details and status records, and the user interface enables citizens and authorized personnel to interact with the system. The proposed solution aims to improve the organization, traceability, and timely handling of civic grievances while strengthening accountability in the complaint resolution process.

4. Problem Statement and Objectives

Problem Statement: Citizens often face difficulties in reporting civic issues, identifying responsible departments, tracking grievance status, and receiving timely resolutions. Manual complaint classification, inefficient departmental coordination, and the absence of systematic escalation can delay civic service delivery and reduce transparency.

Problem Description: Traditional grievance management systems depend heavily on manual complaint registration, categorization, departmental assignment, and follow-up. UrbanGuardian AI aims to address these challenges through an integrated platform that combines AI-assisted grievance analysis, automatic complaint classification, department identification, complaint tracking, and administrative monitoring. The system follows a microservices-based architecture and is designed to support Kafka-based event communication and automated escalation of unresolved grievances.

Objectives:

- **AI-Based Grievance Detection**: Analyze citizens' reported civic issues and identify the appropriate grievance category using Artificial Intelligence.

- **Automatic Complaint Classification:** Categorize grievances related to roads, sanitation, water supply, drainage, streetlights, and other civic services.

- **Smart Department Identification:** Identify the appropriate department for each grievance based on its category and priority.

- **Complaint Tracking and Monitoring:** Enable citizens to register complaints, receive unique tracking identifiers, and monitor their resolution status through a user-friendly interface.

- **Administrative Grievance Management:** Provide an administrative dashboard for reviewing complaints, updating their status, and monitoring resolution progress.

- **AI-Assisted Citizen Support:** Integrate an AI chatbot to provide interactive assistance and guidance regarding civic grievances.

- **Event-Driven Communication and Escalation:** Design the system to support Apache Kafka-based event processing and automated escalation of unresolved complaints.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent processor.
- **RAM:** Minimum 8 GB; 16 GB recommended for running multiple microservices, PostgreSQL, and AI components simultaneously.
- **Storage:** Minimum 20 GB free disk space; SSD recommended for better application performance.
- **System Architecture:** 64-bit processor and operating system**.**
- **Internet Connection:** Stable internet connection for accessing Google Gemini AI services, downloading dependencies, and API communication.
- Display: Minimum 1366 × 768 screen resolution.

Software Requirements:

- **Operating System:** Windows 10/11
- **Programming Languages:** Java 21, Python 3, JavaScript, HTML5, CSS3, SQL
- **Frameworks:** Spring Boot, Spring Cloud, React.js, FastAPI
- **Database**: PostgreSQL
- **AI Technology:** Google Gemini AI and Large Language Models (LLMs)
- **Architecture & Communication:** Microservices, REST APIs, Eureka Service Discovery, Spring Cloud Gateway, Apache Kafka
- **Development Tools:** VS Code, Spring Tool Suite, Maven, Git, GitHub
- **Testing Tools:** Postman

6. IMPLEMENTATION

6.1 Frontend :-

The frontend of UrbanGuardian AI is developed using React.js, HTML, CSS, and JavaScript. It provides an interactive web interface through which citizens can register complaints, describe civic issues, view AI-generated complaint classifications, and track their grievance status using unique tracking identifiers. An administrative dashboard allows authorized personnel to review complaints, monitor their progress, and update resolution status. The frontend communicates with backend services through REST APIs.

The major frontend components include:

- **App.jsx**: Main React application responsible for rendering the user interface and managing application navigation.
- **Login Interface:** Provides login functionality for accessing the platform.
- **Citizen Dashboard:** Allows citizens to submit civic grievances and view complaint information.
- **Complaint Registration:** Collects complaint descriptions and other relevant details.
- **Complaint Tracking:** Displays complaint status using a unique tracking identifier.
- **Admin Dashboard:** Enables administrators to view, manage, and update complaints.
- **AI Chatbot Interface:** Provides interactive assistance to citizens regarding civic grievances.
- **App.css / index.css:** Contains styling, layout definitions, and responsive interface design.

The main frontend application is organized around App.jsx, which integrates the complaint registration, tracking, administrative, and chatbot functionality.

App.jsx :-

```
const API = "/api/complaints";
const initialForm = {
  citizenName: "",
  citizenEmail: "",
  location: "",
  description: ""
};
const submitComplaint = async (event) => {
```

```
  event.preventDefault();
  setLoading(true);
  setMessage("");
  setResult(null);
  try {
    const structured = [
      locationArea === "Other area" ? "" : locationArea,
      locationCity,
      locationState
    ].filter(Boolean).join(", ");
    const location = [
      structured,
      locationLandmark.trim() || form.location.trim()
    ].filter(Boolean).join(" — ");
    if (!location) {
      setMessage("Please choose a location or enter an area/landmark.");
      return;
    }
    const response = await axios.post(API, {
      ...form,
      location
    });

    setResult(response.data);
    setForm(initialForm);
  } catch (error) {
    setMessage("Submission failed: " + errorText(error));
  } finally {
    setLoading(false);
  }
};
const trackComplaint = async (event) => {
  event?.preventDefault();
  if (!trackingCode.trim()) return;
  setLoading(true);
  setMessage("");
  setTracked(null);
```

```
    try {
      const response = await axios.get(
        `${API}/track/${encodeURIComponent(trackingCode.trim())}`
      );
      setTracked(response.data);
    } catch (error) {
      setMessage(
        error.response?.status === 404
          ? "Complaint not found. Check your tracking code."
          : "Tracking failed: " + errorText(error)
      );
    } finally {
      setLoading(false);
    }
  };
```

6.2 Backend :-

The backend is developed using Java, Spring Boot, Spring Cloud, and Python, following a Service-Oriented Architecture (SOA). Independent services handle complaint management, AI analysis, API routing, and service discovery.

Complaint Service

The Complaint Service is responsible for registering, storing, retrieving, and updating citizen grievances.

- **Complaint Entity:** Represents complaint information, including complaint description, category, priority, department, and status.
- **Complaint Repository:** Performs database operations using Spring Data JPA.
- **Complaint Service Logic:** Handles complaint registration, tracking, and status management.
- **Complaint Controller:** Exposes REST endpoints for complaint-related operations.
- **PostgreSQL:** Stores complaint details and associated tracking information.

AI Analysis Service

The AI Analysis Service supports intelligent processing of citizen complaint descriptions.

- **AI Analysis Controller:** Receives complaint information for analysis.

- **AI Analysis Request:** Represents the input data required for complaint analysis.
- **Classification Logic:** Identifies civic issue categories, priority levels, and responsible departments.
- **AI Analysis Response:** Returns the classification results to the complaint management workflow.

API Gateway

The API Gateway is implemented using Spring Cloud Gateway and acts as the centralized entry point for client requests. It routes requests to the appropriate backend services, reducing direct dependencies between the frontend and individual microservices.

Eureka Server

Eureka Server provides service registration and discovery. It enables the API Gateway and backend services to identify available service instances and supports communication within the microservices architecture.

Python AI Agent Service

The Python-based AI service provides additional intelligent functionality using Google Gemini AI.

- **main.py:** Contains the main AI service application and its API functionality.
- **chatbot_addon.py:** Supports AI-assisted conversational interactions for citizen guidance.
- **Google Gemini AI:** Provides large language model capabilities for processing user queries and generating contextual responses.

ComplaintService.java:-

```
@Service
public class ComplaintService {
   private final ComplaintRepository complaintRepository;
   private final AiAnalysisClient aiAnalysisClient;
   private final ComplaintEventProducer eventProducer;
   public ComplaintService(
         ComplaintRepository complaintRepository,
```

```
        AiAnalysisClient aiAnalysisClient,
        ComplaintEventProducer eventProducer) {
    this.complaintRepository = complaintRepository;
    this.aiAnalysisClient = aiAnalysisClient;
    this.eventProducer = eventProducer;
}
Public Complaint createComplaint(Complaint complaint) {
    String complaintCode = "UG-" + UUID.randomUUID()
            .toString().substring(0, 8).toUpperCase();
    complaint.setComplaintCode(complaintCode);
    complaint.setStatus("SUBMITTED");
    complaint.setEscalated(false);
    try {
        AiAnalysisClient.AiAnalysisResult analysis =
                aiAnalysisClient.analyzeComplaint(
                        complaint.getDescription(),
                        complaint.getLocation());
        if (analysis != null) {
            complaint.setCategory(analysis.category());
            complaint.setPriority(analysis.priority());
            complaint.setDepartment(analysis.department());
            complaint.setAiReason(analysis.reason());
            complaint.setAiSummary(analysis.summary());
            complaint.setSuggestedAction(
                    analysis.suggestedAction());
            complaint.setNeedsHumanReview(
```

```
                analysis.needsHumanReview());
        }
    } catch (Exception e) {
        complaint.setCategory("PENDING_ANALYSIS");
        complaint.setPriority("MEDIUM");
        complaint.setDepartment("Municipal Administration");
        complaint.setAiReason(
                "AI analysis is temporarily unavailable. "
                + "Complaint saved for later analysis.");
        complaint.setAiSummary(null);
        complaint.setSuggestedAction(null);
        complaint.setNeedsHumanReview(true);
    }
    Complaint saved = complaintRepository.save(complaint);
    eventProducer.publish(
            "COMPLAINT_CREATED",
            saved.getComplaintCode(),
            saved.getPriority());
    return saved;
}
// Complaint tracking and status updates
public List<Complaint> getAllComplaints() {
    return complaintRepository.findAll();
}
// Get complaint by ID
public Optional<Complaint> getComplaintById(Long id) {
```

```
    return complaintRepository.findById(id);
}
// Track complaint using unique code
public Optional<Complaint> getComplaintByCode(
        String complaintCode) {
    return complaintRepository
            .findByComplaintCode(complaintCode);
}
// Update complaint status
public Optional<Complaint> updateStatus(
        Long id, String status) {
    return complaintRepository.findById(id)
            .map(complaint -> {
                complaint.setStatus(status);
                if ("IN_PROGRESS".equals(status)
                        || "RESOLVED".equals(status)) {
                    complaint.setEscalated(false);
                }
                Complaint saved =
                        complaintRepository.save(complaint);
                eventProducer.publish(
                        "COMPLAINT_STATUS_UPDATED",
                        saved.getComplaintCode(),
                        saved.getStatus());
                return saved;
            });
```

}

6.3 API Integration :-

The frontend and backend components are integrated using RESTful APIs. The API Gateway routes incoming requests to the appropriate microservices, while Eureka supports service discovery.

- **React Frontend:** Sends HTTP requests for complaint registration, tracking, and administrative operations.
- **API Gateway:** Provides a centralized entry point for backend requests.
- **Eureka Server:** Supports service discovery and registration.
- **REST Controllers:** Expose endpoints for complaint management and AI analysis.
- **PostgreSQL:** Maintains persistent complaint and status information.
- **Python AI Service:** Supports AI-powered chatbot functionality and intelligent assistance.
- **Apache Kafka:** Forms part of the proposed event-driven architecture for complaint updates, notifications, and escalation workflows.

API Flow:

React Frontend → API Gateway → Complaint Service → PostgreSQL Database

AI Analysis Flow:

Complaint Service → AI Analysis Service → Complaint Classification and Department Identification

AI Chatbot Flow:

React Frontend → Python AI Service → Google Gemini AI → Chatbot Response

Proposed Event-Driven Flow:

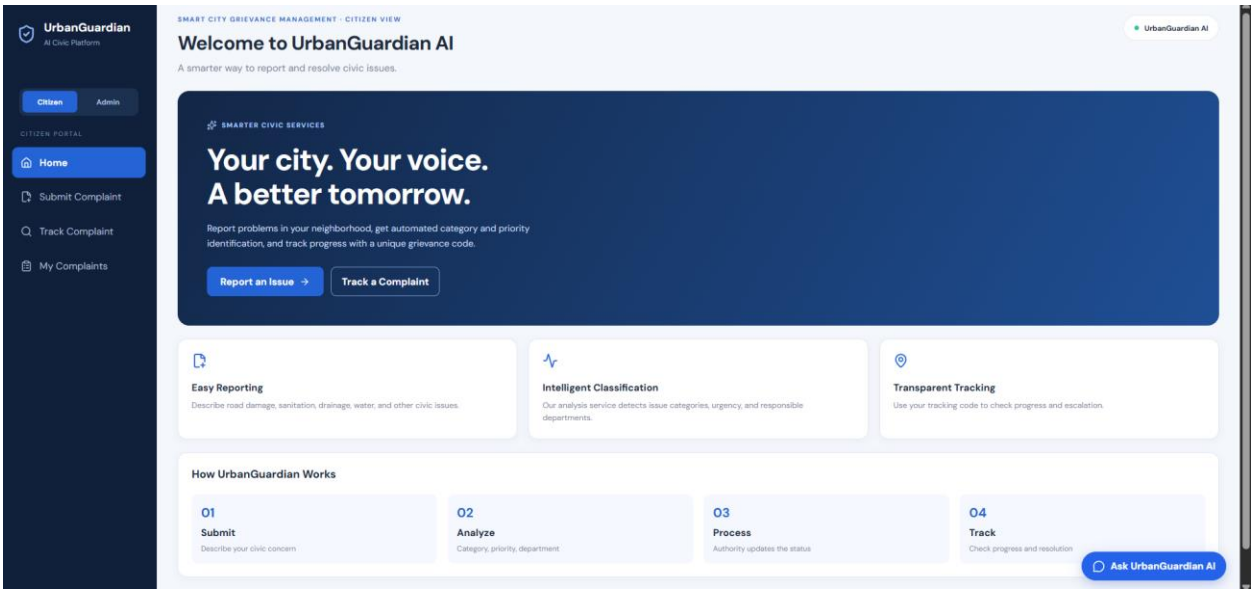
Complaint Service → Apache Kafka → Notification / Escalation Service

The integration enables citizens to submit and track grievances while allowing administrators to manage complaint resolution through a centralized platform. The microservices architecture provides modularity and a foundation for future enhancements such as asynchronous event processing, automated escalation, and advanced grievance analytics.

7. RESULTS & DISCUSSION

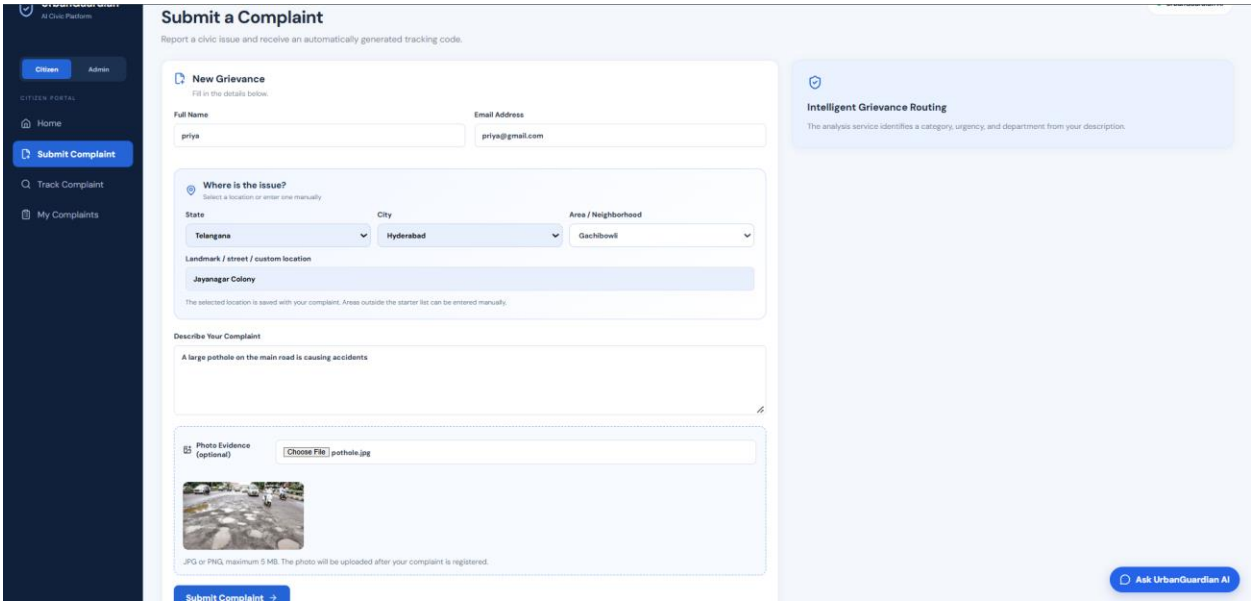
The developed UrbanGuardian AI platform integrates citizen grievance registration, AI-assisted complaint classification, department identification, complaint tracking, administrative monitoring, and automated escalation through a microservices-based architecture.

7.1 Citizen Dashboard



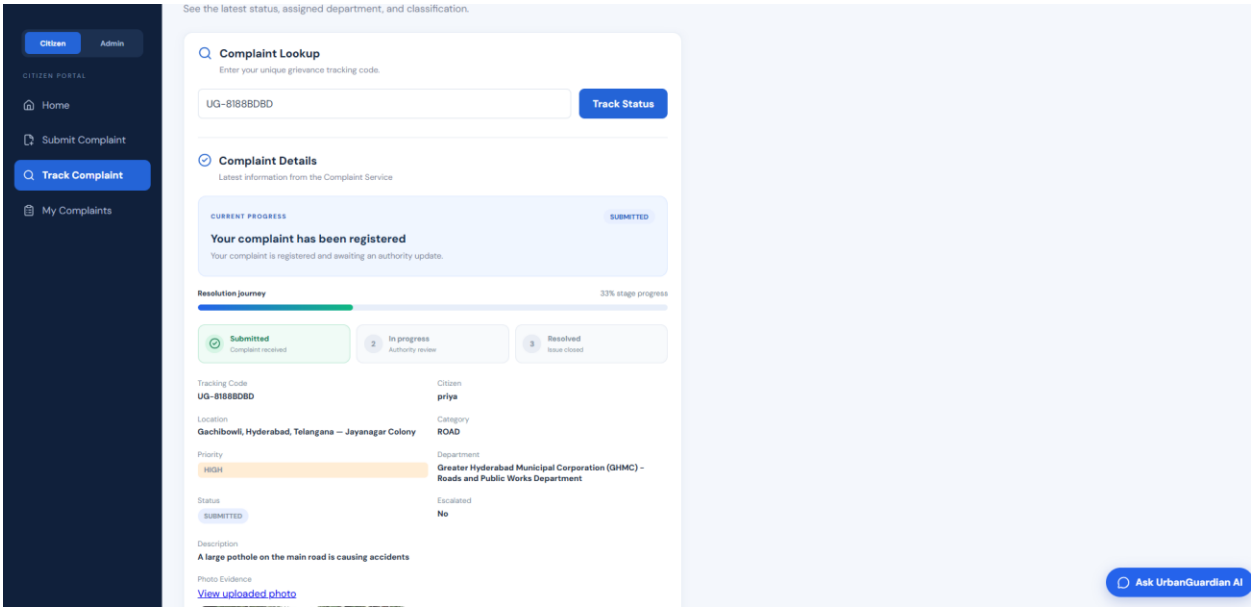
The citizen dashboard provides a user-friendly interface for accessing grievance registration, complaint tracking, and other civic services.

7.2 Complaint Registration Page



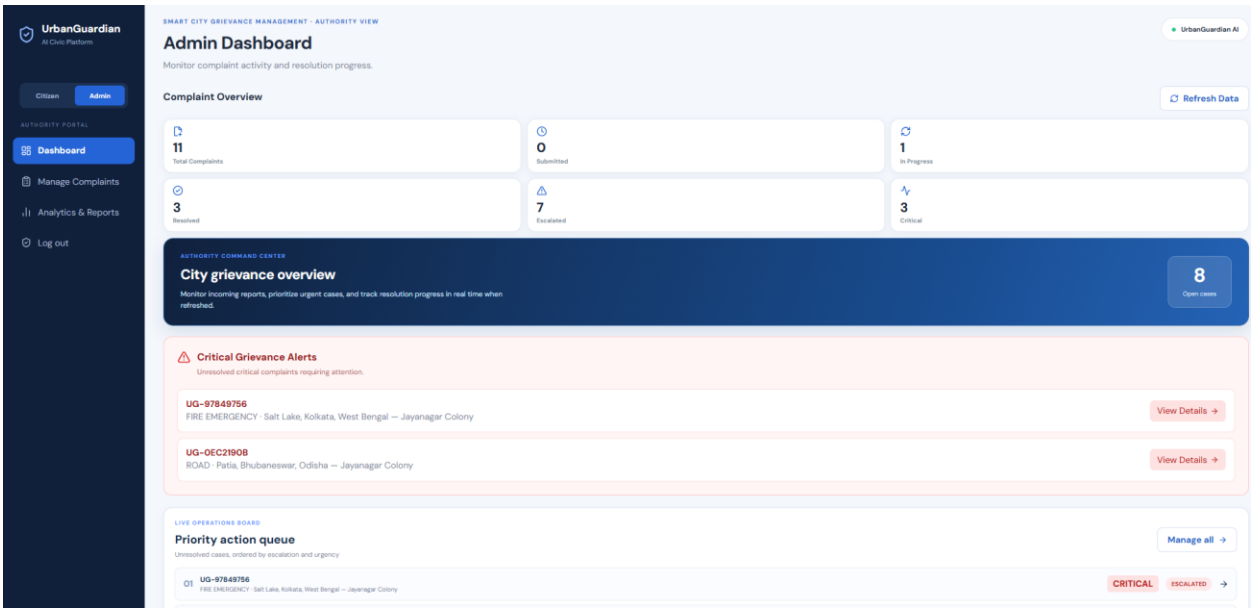
The complaint registration page allows citizens to submit civic issues by entering their details, location, and complaint description.

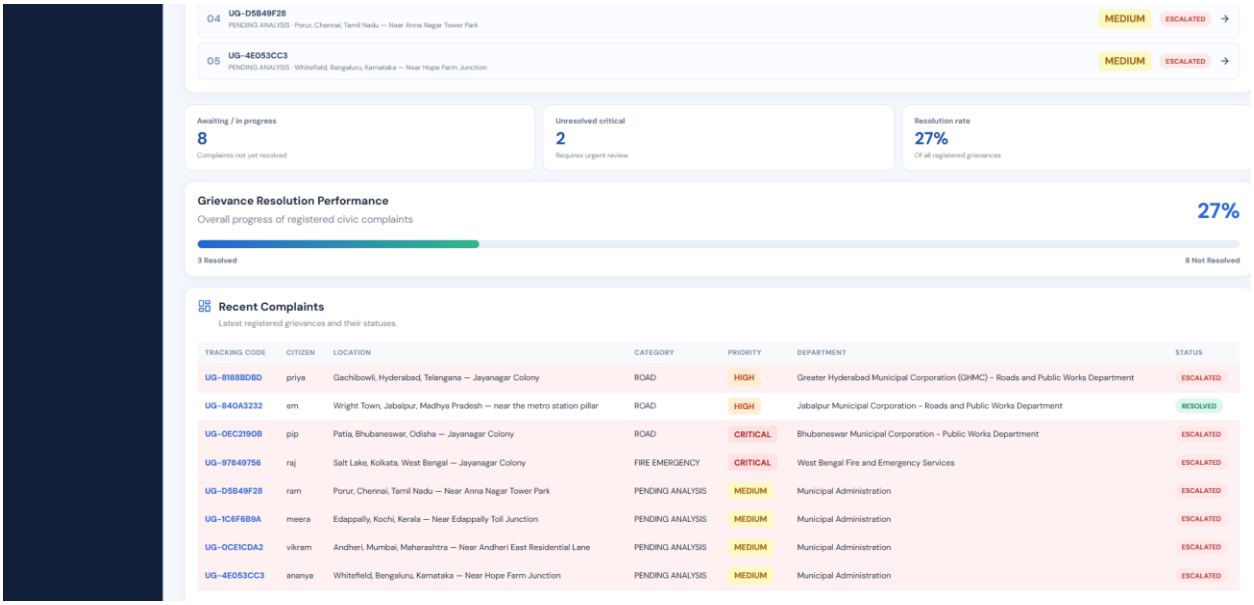
7.3 Complaint Tracking Page



The complaint tracking page allows citizens to check the progress of their grievances using a unique tracking code. It displays the complaint details, assigned department, priority, and current resolution status.

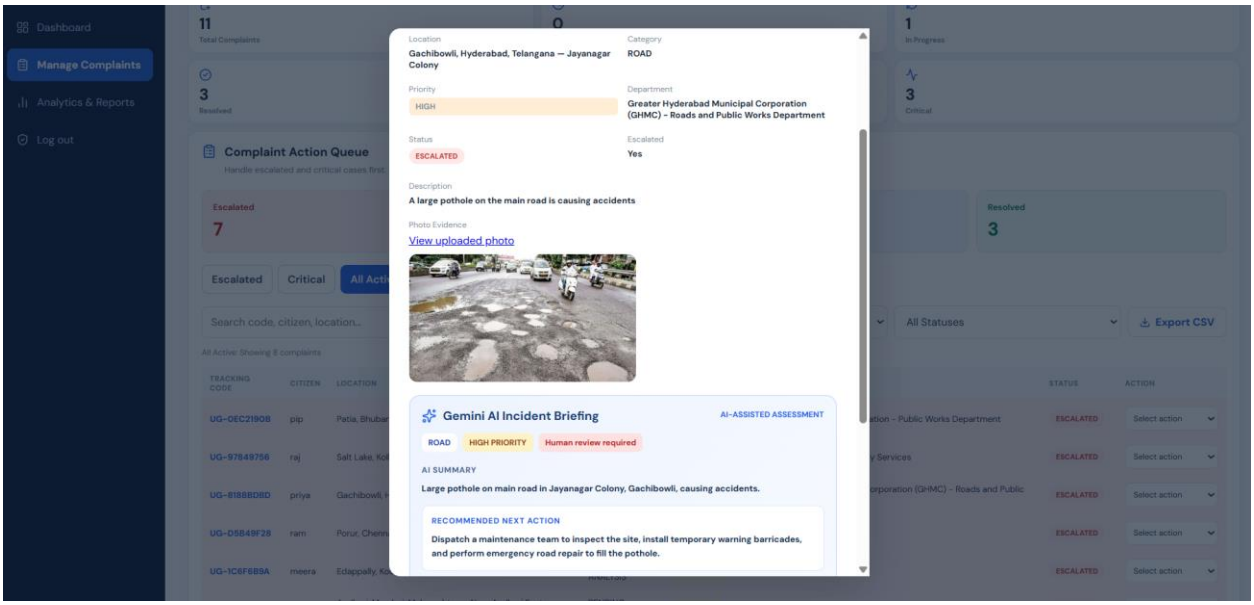
7.4 Admin Dashboard





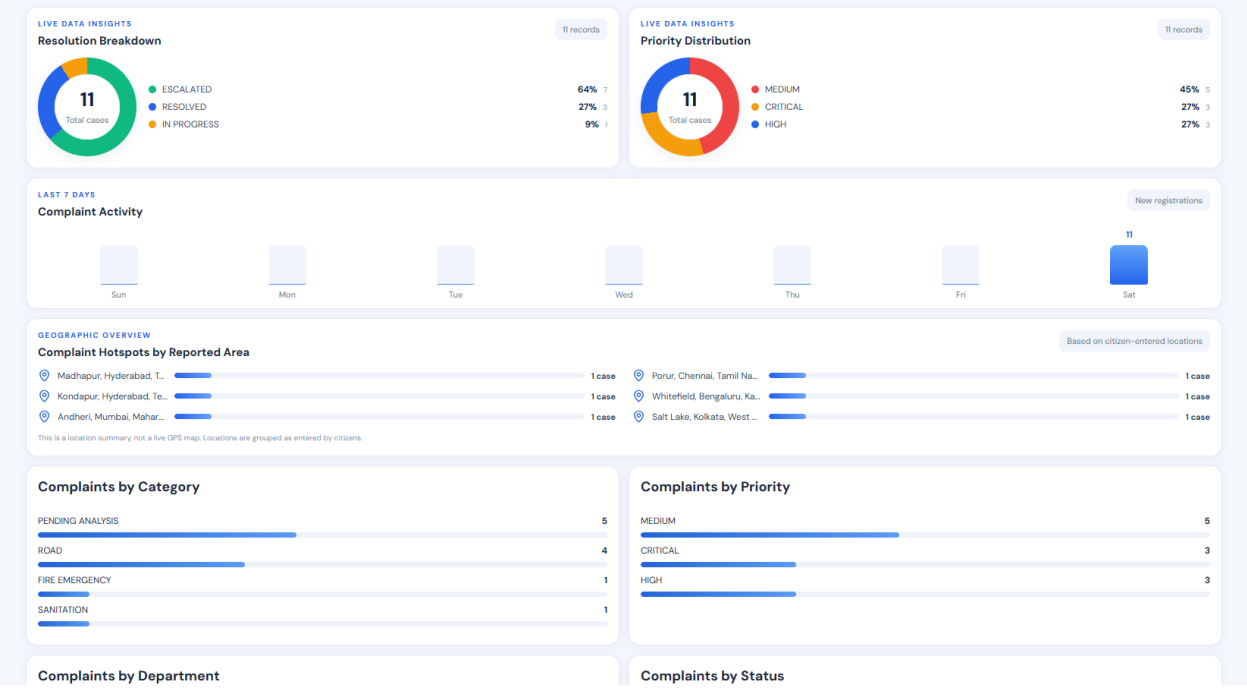
The admin dashboard provides a centralized view of registered complaints, including submitted, in-progress, resolved, and escalated grievances. Administrators can monitor complaints and update their resolution status.

7.5 AI-Based Complaint Analysis



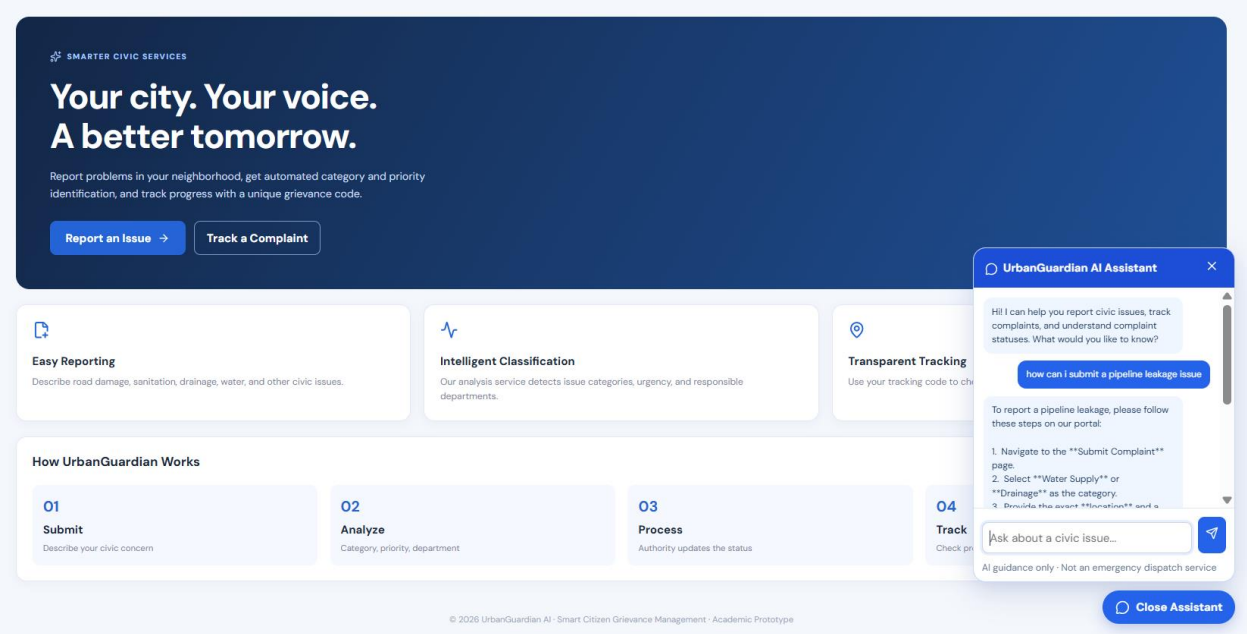
The system analyzes complaint descriptions and identifies the complaint category, priority level, and responsible department.

7.6 Complaint Analytics Page



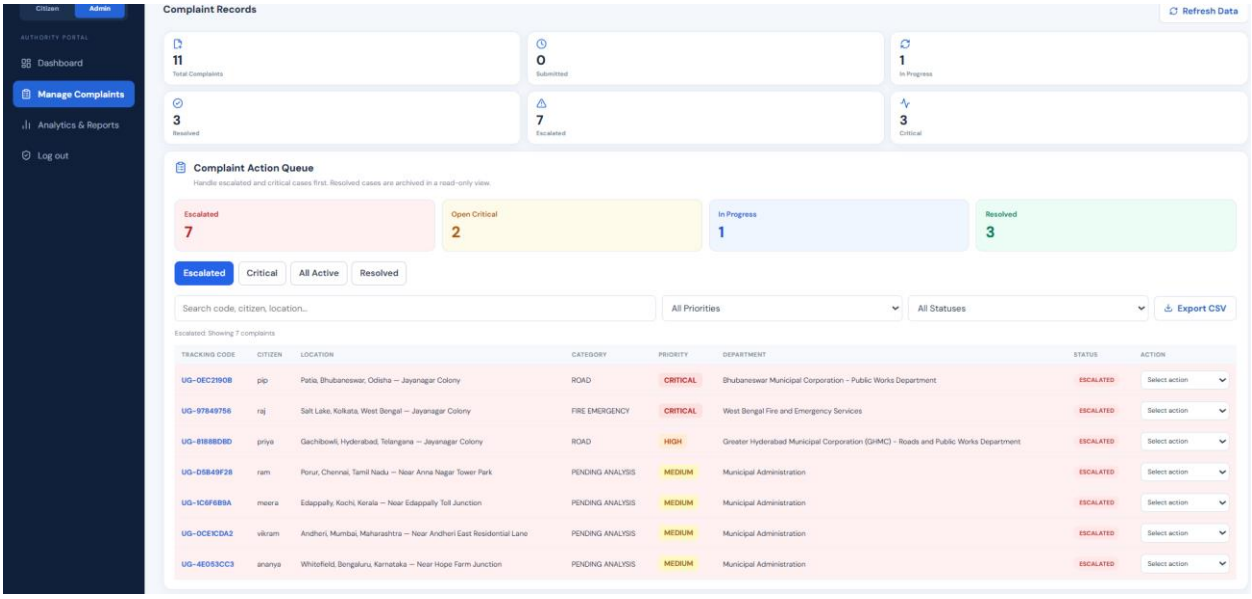
The analytics dashboard presents complaint statistics based on categories, priorities, locations, and resolution status. These insights help administrators identify frequently reported civic issues and monitor grievance resolution performance.

7.7 AI Chatbot



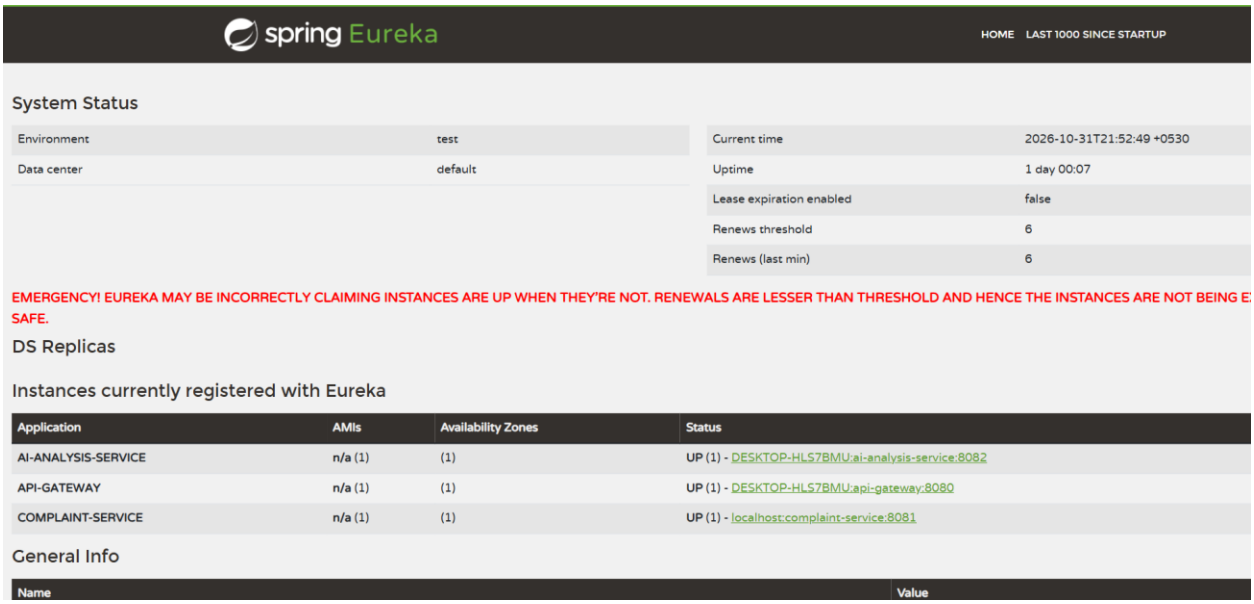
The AI chatbot provides interactive assistance to citizens regarding civic issues, complaint registration, and tracking procedures.

7.8 Automated Complaint Escalation



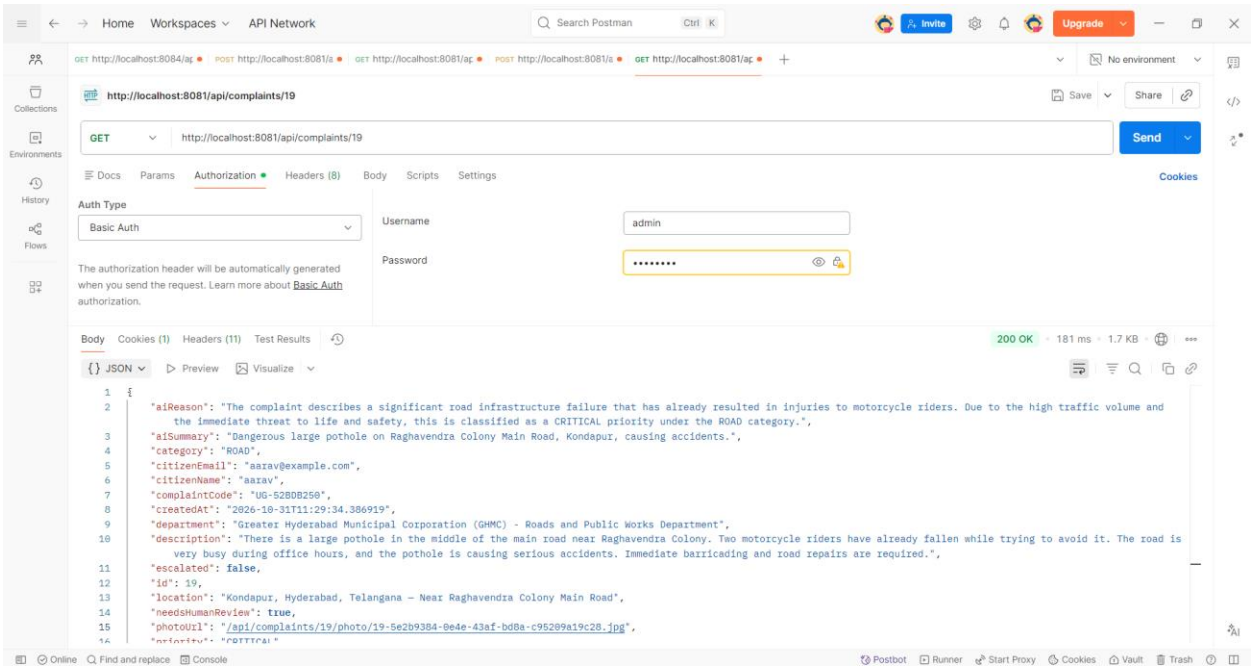
The system automatically identifies overdue complaints and marks them as escalated when the configured time limit is exceeded.

7.9 Eureka Service Dashboard



The Eureka dashboard displays the registered microservices and their running status.

7.10 API Testing



The REST APIs are tested using Postman to verify complaint registration, complaint retrieval, tracking, and communication between backend services. Successful API responses demonstrate the functionality of the implemented endpoints.

8. CONCLUSION & FUTURE SCOPE

Conclusion

UrbanGuardian AI is an intelligent citizen grievance management platform developed to simplify the process of reporting civic issues and improving their resolution. The system uses Artificial Intelligence (AI) to identify and classify grievances related to problems such as damaged roads, improper waste disposal, water supply disruptions, drainage issues, and non-functional streetlights. By integrating AI-based grievance detection with Apache Kafka, the platform supports organized complaint processing, departmental routing, status tracking, and automated escalation of unresolved complaints. The backend manages grievance workflows, while the database maintains complaint details and resolution records. Overall, UrbanGuardian AI aims to reduce delays in grievance handling, improve coordination between departments, and strengthen transparency and accountability in civic service delivery.

Future Scope

1. Short-Term Enhancements

- Improve AI-based grievance detection and classification accuracy.
- Enhance complaint tracking and provide clearer status notifications to citizens.
- Improve the user interface for complaint registration and grievance monitoring.

2. Medium-Term Enhancements

- Implement priority-based grievance handling according to complaint severity.
- Enhance automated escalation rules based on predefined resolution deadlines.
- Introduce dashboards and analytical reports to monitor departmental performance and unresolved complaints.
- Integrate location-based complaint reporting to help identify the affected areas accurately.

3. Long-Term Enhancements

- Integrate image-based grievance detection to identify civic issues from uploaded photographs.
- Develop predictive analytics to identify recurring civic problems and support preventive maintenance.
- Expand the platform to support multiple municipalities and additional categories of public grievances.

- Introduce multilingual support and AI-powered conversational assistance to make grievance reporting more accessible to citizens.
- These enhancements can help UrbanGuardian AI evolve into a more intelligent, scalable, and comprehensive citizen grievance management platform.

9. REFERENCES

1. React.js Documentation: https://react.dev/
2. Spring Boot Documentation: https://spring.io/projects/spring-boot
3. Spring Cloud Documentation: https://spring.io/projects/spring-cloud
4. Spring AI Documentation: https://docs.spring.io/spring-ai/reference/
5. Ollama Documentation: https://docs.ollama.com/
6. PostgreSQL Documentation: https://www.postgresql.org/docs/
7. Apache Kafka Documentation: https://kafka.apache.org/documentation/
8. Spring Security Documentation: https://spring.io/projects/spring-security
9. Docker Documentation: https://docs.docker.com/
10. JSON Web Tokens (JWT): https://jwt.io/introduction
11. Swachhata-MoHUA: https://swachhatahiseva.com/
12. MyGov India: https://www.mygov.in/
13. Centralized Public Grievance Redress and Monitoring System (CPGRAMS): https://pgportal.gov.in/
14. Postman Documentation: https://learning.postman.com/docs/